



Diff_Materialize

— 2 removals

131 lines

```

1 # main.py
2 # Build a single, ever-growing CSV from
  all structured JSONL files.
3 # Reads:
  gs://<bucket>/<STRUCTURED_PREFIX>/run_i
  d=*/jsonl/*.jsonl
4 # Writes:
  gs://<bucket>/<STRUCTURED_PREFIX>/datas
  ets/listings_master.csv (atomic
  publish)
5
6 import csv
7 import io
8 import json
9 import os
10 import re
11 from datetime import datetime, timezone
12 from typing import Dict, Iterable
13
14 from flask import Request, jsonify
15 from google.cloud import storage
16
17 # ----- ENV -----
  -----
18 BUCKET_NAME      =
  os.getenv("GCS_BUCKET")
  # REQUIRED
19 STRUCTURED_PREFIX =
  os.getenv("STRUCTURED_PREFIX",
  "structured") # e.g., "structured"
20
21 storage_client = storage.Client()
22
23 # Accept BOTH runIDs:
24 RUN_ID_ISO_RE    =
  re.compile(r"^\d{8}T\d{6}Z$") #
  20251026T170002Z
25 RUN_ID_PLAIN_RE =
  re.compile(r"^\d{14}$")      #

```

+ 3 additions

131 lines

```

1 # main.py
2 # Build a single, ever-growing CSV from
  all structured JSONL files.
3 # Reads:
  gs://<bucket>/<STRUCTURED_PREFIX>/run_i
  d=*/jsonl/*.jsonl
4 # Writes:
  gs://<bucket>/<STRUCTURED_PREFIX>/datas
  ets/listings_master.csv (atomic
  publish)
5
6 import csv
7 import io
8 import json
9 import os
10 import re
11 from datetime import datetime, timezone
12 from typing import Dict, Iterable
13
14 from flask import Request, jsonify
15 from google.cloud import storage
16
17 # ----- ENV -----
  -----
18 BUCKET_NAME      =
  os.getenv("GCS_BUCKET")
  # REQUIRED
19 STRUCTURED_PREFIX =
  os.getenv("STRUCTURED_PREFIX",
  "structured") # e.g., "structured"
20
21 storage_client = storage.Client()
22
23 # Accept BOTH runIDs:
24 RUN_ID_ISO_RE    =
  re.compile(r"^\d{8}T\d{6}Z$") #
  20251026T170002Z
25 RUN_ID_PLAIN_RE =
  re.compile(r"^\d{14}$")      #

```

```

20251026170002
26
27 # Stable CSV schema for students
28 CSV_COLUMNS = [
29     "post_id", "run_id", "scraped_at",
30     "price", "year", "make", "model",
31     "mileage",
32     "source_txt"
33 ]
34 def _list_run_ids(bucket: str,
35     structured_prefix: str) -> list[str]:
36     it =
37     storage_client.list_blobs(bucket,
38     prefix=f"{structured_prefix}/",
39     delimiter="/")
40     for _ in it: # populate
41     it.prefixes
42     pass
43     run_ids = []
44     for p in getattr(it, "prefixes",
45     []):
46         tail = p.rstrip("/").split("/")
47         [-1] # e.g.
48         run_id=20251026170002
49         if tail.startswith("run_id="):
50             rid = tail.split("run_id=",
51             1)[1]
52             if RUN_ID_ISO_RE.match(rid)
53             or RUN_ID_PLAIN_RE.match(rid):
54                 run_ids.append(rid)
55     return sorted(run_ids)
56
57 def _jsonl_records_for_run(bucket: str,
58     structured_prefix: str, run_id: str):
59     """Yield dict records from .jsonl
60     under .../run_id=<run_id>/jsonl/ (one
61     JSON per file)."""
62     b = storage_client.bucket(bucket)
63     prefix = f"
64     {structured_prefix}/run_id=
65     {run_id}/jsonl/"
66     for blob in
67     b.list_blobs(prefix=prefix):
68         if not
69         blob.name.endswith(".jsonl"):
70             continue

```

```

20251026170002
26
27 # Stable CSV schema for students
28 CSV_COLUMNS = [
29     "post_id", "run_id", "scraped_at",
30     "price", "year", "make", "model",
31     "mileage", "transmission",
32     "source_txt"
33 ]
34 def _list_run_ids(bucket: str,
35     structured_prefix: str) -> list[str]:
36     it =
37     storage_client.list_blobs(bucket,
38     prefix=f"{structured_prefix}/",
39     delimiter="/")
40     for _ in it: # populate
41     it.prefixes
42     pass
43     run_ids = []
44     for p in getattr(it, "prefixes",
45     []):
46         tail = p.rstrip("/").split("/")
47         [-1] # e.g.
48         run_id=20251026170002
49         if tail.startswith("run_id="):
50             rid = tail.split("run_id=",
51             1)[1]
52             if RUN_ID_ISO_RE.match(rid)
53             or RUN_ID_PLAIN_RE.match(rid):
54                 run_ids.append(rid)
55     return sorted(run_ids)
56
57 def _jsonl_records_for_run(bucket: str,
58     structured_prefix: str, run_id: str):
59     """Yield dict records from .jsonl
60     under .../run_id=<run_id>/jsonl/ (one
61     JSON per file)."""
62     b = storage_client.bucket(bucket)
63     prefix = f"
64     {structured_prefix}/run_id=
65     {run_id}/jsonl_llm/"
66     for blob in
67     b.list_blobs(prefix=prefix):
68         if not
69         blob.name.endswith(".jsonl"):
70             continue

```

```

54         data = blob.download_as_text()
55         line = data.strip()
56         if not line:
57             continue
58         try:
59             rec = json.loads(line)
60             # ensure required keys
        exist
61             rec.setdefault("run_id",
run_id)
62             yield rec
63         except Exception:
64             continue
65
66 def _run_id_to_dt(rid: str) ->
datetime:
67     if RUN_ID_ISO_RE.match(rid):
68         return datetime.strptime(rid,
"%Y%m%dT%H%M%SZ").replace(tzinfo=timezone.utc)
69     if RUN_ID_PLAIN_RE.match(rid):
70         return datetime.strptime(rid,
"%Y%m%d%H%M%S").replace(tzinfo=timezone.utc)
71     # fallback: now
72     return datetime.now(timezone.utc)
73
74 def _open_gcs_text_writer(bucket: str,
key: str):
75     """Open a text-mode writer to GCS;
close() will finalize the upload."""
76     b = storage_client.bucket(bucket)
77     blob = b.blob(key)
78     # Text mode avoids the
flush/finalize pitfall of
binary+TextIOWrapper
79     return blob.open("w") # newline
handled by csv module
80
81
82 def _write_csv(records: Iterable[Dict],
dest_key: str, columns=CSV_COLUMNS) ->
int:
83     n = 0
84     with
_open_gcs_text_writer(BUCKET_NAME,
dest_key) as out:
85         w = csv.DictWriter(out,
fieldnames=columns,

```

```

54         data = blob.download_as_text()
55         line = data.strip()
56         if not line:
57             continue
58         try:
59             rec = json.loads(line)
60             # ensure required keys
        exist
61             rec.setdefault("run_id",
run_id)
62             yield rec
63         except Exception:
64             continue
65
66 def _run_id_to_dt(rid: str) ->
datetime:
67     if RUN_ID_ISO_RE.match(rid):
68         return datetime.strptime(rid,
"%Y%m%dT%H%M%SZ").replace(tzinfo=timezone.utc)
69     if RUN_ID_PLAIN_RE.match(rid):
70         return datetime.strptime(rid,
"%Y%m%d%H%M%S").replace(tzinfo=timezone.utc)
71     # fallback: now
72     return datetime.now(timezone.utc)
73
74 def _open_gcs_text_writer(bucket: str,
key: str):
75     """Open a text-mode writer to GCS;
close() will finalize the upload."""
76     b = storage_client.bucket(bucket)
77     blob = b.blob(key)
78     # Text mode avoids the
flush/finalize pitfall of
binary+TextIOWrapper
79     return blob.open("w") # newline
handled by csv module
80
81
82 def _write_csv(records: Iterable[Dict],
dest_key: str, columns=CSV_COLUMNS) ->
int:
83     n = 0
84     with
_open_gcs_text_writer(BUCKET_NAME,
dest_key) as out:
85         w = csv.DictWriter(out,
fieldnames=columns,

```

```

        extrasaction="ignore")
86         w.writeheader()
87         for rec in records:
88             row = {c: rec.get(c, None)
for c in columns}
89             w.writerow(row)
90             n += 1
91         return n # close() finalizes the
upload
92
93 def materialize_http(request: Request):
94     """
95     HTTP POST (no body needed).
96     Crawls ALL structured run folders,
de-dupes by post_id (keep newest run),
97     and writes one CSV directly to
.../datasets/listings_master.csv.
98     Returns JSON with counts and output
path.
99     """
100     try:
101         if not BUCKET_NAME:
102             return jsonify({"ok":
False, "error": "missing GCS_BUCKET
env"}), 500
103
104         run_ids =
_list_run_ids(BUCKET_NAME,
STRUCTURED_PREFIX)
105         if not run_ids:
106             return jsonify({"ok":
False, "error": f"no runs found under
{STRUCTURED_PREFIX}/"}, 200
107
108         latest_by_post: Dict[str, Dict]
= {}
109         for rid in run_ids:
110             for rec in
_jsonl_records_for_run(BUCKET_NAME,
STRUCTURED_PREFIX, rid):
111                 pid =
rec.get("post_id")
112                 if not pid:
113                     continue
114                 prev =
latest_by_post.get(pid)
115                 if (prev is None) or
(_run_id_to_dt(rec.get("run_id", rid))

```

```

        extrasaction="ignore")
86         w.writeheader()
87         for rec in records:
88             row = {c: rec.get(c, None)
for c in columns}
89             w.writerow(row)
90             n += 1
91         return n # close() finalizes the
upload
92
93 def materialize_http(request: Request):
94     """
95     HTTP POST (no body needed).
96     Crawls ALL structured run folders,
de-dupes by post_id (keep newest run),
97     and writes one CSV directly to
.../datasets/listings_master.csv.
98     Returns JSON with counts and output
path.
99     """
100     try:
101         if not BUCKET_NAME:
102             return jsonify({"ok":
False, "error": "missing GCS_BUCKET
env"}), 500
103
104         run_ids =
_list_run_ids(BUCKET_NAME,
STRUCTURED_PREFIX)
105         if not run_ids:
106             return jsonify({"ok":
False, "error": f"no runs found under
{STRUCTURED_PREFIX}/"}, 200
107
108         latest_by_post: Dict[str, Dict]
= {}
109         for rid in run_ids:
110             for rec in
_jsonl_records_for_run(BUCKET_NAME,
STRUCTURED_PREFIX, rid):
111                 pid =
rec.get("post_id")
112                 if not pid:
113                     continue
114                 prev =
latest_by_post.get(pid)
115                 if (prev is None) or
(_run_id_to_dt(rec.get("run_id", rid))

```

```

> _run_id_to_dt(prev.get("run_id",
""))):
116         latest_by_post[pid]
= rec
117
118         base = f"
{STRUCTURED_PREFIX}/datasets"
119         final_key = f"
{base}/listings_master.csv"
120         rows =
_write_csv(latest_by_post.values(),
final_key)
121
122         return jsonify({
123             "ok": True,
124             "runs_scanned":
len(run_ids),
125             "unique_listings":
len(latest_by_post),
126             "rows_written": rows,
127             "output_csv":
f"gs://{BUCKET_NAME}/{final_key}"
128         }), 200
129     except Exception as e:
130         # Return a JSON error so you
don't just see a plain 500
131         return jsonify({"ok": False,
"error": f"{type(e).__name__}: {e}"})
500

```

```

> _run_id_to_dt(prev.get("run_id",
""))):
116         latest_by_post[pid]
= rec
117
118         base = f"
{STRUCTURED_PREFIX}/datasets"
119         final_key = f"
{base}/listings_master_llm.csv"
120         rows =
_write_csv(latest_by_post.values(),
final_key)
121
122         return jsonify({
123             "ok": True,
124             "runs_scanned":
len(run_ids),
125             "unique_listings":
len(latest_by_post),
126             "rows_written": rows,
127             "output_csv":
f"gs://{BUCKET_NAME}/{final_key}"
128         }), 200
129     except Exception as e:
130         # Return a JSON error so you
don't just see a plain 500
131         return jsonify({"ok": False,
"error": f"{type(e).__name__}: {e}"})
500

```